MSDS 682 · DATA STREAM PROCESSING

Space Weather Radio-Risk Monitor

A reproducible Kafka alert when planetary Kp crosses an elevated-risk threshold.

Noam Fein · Niki Naderzad

PROBLEM AND USEFUL RESULT

A radio operator needs a signal, not another feed to watch

Kp ≥ 6

Our course-scale threshold for elevated radio-risk conditions.

Emit one alert when the rolling 15-minute maximum first reaches or exceeds 6.

- Target user: amateur-radio operators relying on HF propagation.
- No repeated alerts while the window remains elevated.
- The rule rearms only after the rolling maximum falls below 6.

DATA AND EVENT CONTRACT

Every NOAA observation becomes one ordered Kafka event

Source: NOAA SWPC one-minute estimated planetary K-index JSON.

Topic: `kp_observations`
Key: `planetary_kp`

Validation: UTC timestamps, Kp from 0-9, approved provenance, exact four-field contract.

The alert demo uses a clearly labeled synthetic replay because live conditions were quiet. A committed raw NOAA sample verifies the source shape.


```json
{
  "time_tag": "2026-08-11T00:10:00Z",
  "kp_value": 6.3,
  "source": "synthetic://kp-threshold-fixture",
  "ingested_at": "2026-08-11T00:10:05Z"
}
```


Representative record: the first event that creates an alert.

IMPLEMENTED ARCHITECTURE

Kafka separates validated ingestion from stateful alert logic

Replay
9 labeled JSONL events
including 1 duplicate

→

Producer
Validate first
publish in event order

→

Kafka
1 topic · 1 partition
constant global key

→

Consumer
Deduplicate
15-minute rolling max
crossing + rearm

→

Evidence
alert.json · metrics.csv
briefing.txt · evaluation.json

Why this architecture: Kp is one global ordered series, so one partition and one constant key preserve sequence. Kafka keeps the producer and consumer independent and makes replay and duplicates observable.

> **Example:** Kp 6.3 arrives → rolling 15-minute maximum becomes 6.3 → first crossing of Kp ≥ 6 → alert #1

LIVE DEMONSTRATION

Live demo: one command, two alerts

A deterministic replay proves crossing, suppression, rearming, and a second alert.

1 · Kafka replay

`./run_demo.sh` publishes 9 ordered events through local Kafka.

2 · Two crossings

1 duplicate is skipped; the monitor rearms and emits exactly 2 alerts.

3 · Evidence UI

Trace alert #1, the metrics, and the bounded natural-language briefing.

The alert path is deterministic; AI only explains the result

9

Kafka messages processed, including one deliberate duplicate.

2

Expected threshold-crossing alerts, with no alert spam.

7/7

Evaluation assertions passed for alerts and AI briefing cases.

Deterministic code decides

Rolling maximum · threshold · label · alert · fallback.

→

Optional AI phrases five facts

Two sentences only. Changed numbers, labels, causes, places, or advice are rejected.

Verified: 167 standard tests; all 3 Docker-backed Kafka integration tests also passed separately.

Deterministic replay made the streaming behavior provable

WHAT WE LEARNED

Quiet live data could not demonstrate a threshold crossing. A labeled replay reliably proves crossing, deduplication, window expiry, and rearming.

LIMITATION

This is a recent global Kp condition - not a forecast or station-level propagation model.

REALISTIC NEXT STEP

Add a long-running consumer that continuously processes live NOAA events and refreshes the UI.

The result is small, explainable, and reproducible: one ordered stream, one clear rule, and evidence for every alert.

BACKUP · USE ONLY IF THE LIVE DEMO FAILS

Saved terminal evidence

```
$ ./run_demo.sh
Published 9 event(s) · Topic=kp_observations · key=planetary_kp
Processed 9 valid event(s); emitted 2 alert(s)
Briefing source: fallback
Evaluation: 7/7 assertions passed
Artifacts match the labeled fixture:
  2 alerts · 9 metrics rows · briefing present
167 passed, 3 opt-in tests skipped
Demo complete.

Artifacts
  outputs/alert.json
  outputs/metrics.csv
  outputs/briefing.txt
  evaluation/evaluation.json
```